


**A Course End Project Report on**

# Sports Arena Management System-An Integrated Sports Tournament Management System

**A9512- DATABASE MANAGEMENT SYSTEM LABORATORY**

**Submitted by**


| | |
|---|---|
| L.Vishwa Charan | (25881A0560) |
| M.Rohith | (25881A0541) |
| B.Rohit | (25881A0540) |


**Course Facilitator**


Ms. Shaik Shahanaz
Assistant Professor



**Department of Computer Science and Engineering**

**Vardhaman College of Engineering, Hyderabad**


**October 2026**

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

26 Years of Vibrant Vardhaman

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Sports Arena Management System- An Integrated Sports Tournament Management System"** is carried out by **"L.Vishwa Charan", "M.Rohith"** and **"B.Rohit" with** Roll Numbers **"25881A0560" , "25881A0541"** and **"25881A0540"** towards **A9512 – Database Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

| Course Facilitator | Academic Head | Head of the Department |
|---|---|---|
| **Ms. Shaik Shahanaz** | **Dr. R. Karthikeyan** | **Prof. Y. Vijayalata** |
| Assistant Professor, CSE | Professor, CSE | Department of CSE |

# Contents


**1 Introduction** **5**

**2 Problem Analysis** **6**
- 2.1 Problem Statement . . . . . 6
- 2.2 Users and Assumptions . . . . . 6
- 2.3 Functional Requirements . . . . . 7

**3 ER Model** **7**
- 3.1 Entities and Attributes . . . . . 7
- 3.2 Relationships . . . . . 8
- 3.3 Cardinality and Participation . . . . . 9
- 3.4 ER Diagram . . . . . 10

**4 Relational Model** **10**
- 4.1 Relational Schema . . . . . 10
- 4.2 Primary Keys and Foreign Keys . . . . . 11

**5 Normalization** **11**
- 5.1 Functional Dependencies . . . . . 12
- 5.2 First Normal Form (1NF) . . . . . 12
- 5.3 Second Normal Form (2NF) . . . . . 13
- 5.4 Third Normal Form (3NF) . . . . . 13

**6 SQL DDL Commands** **15**
- 6.1 Table Creation . . . . . 15
- 6.2 Constraints . . . . . 17
- 6.3 ALTER Operation . . . . . 17
- 6.4 DROP Operation . . . . . 17

**7 SQL DML Commands** **18**
- 7.1 INSERT . . . . . 18
- 7.2 UPDATE . . . . . 19
- 7.3 DELETE . . . . . 19

**8 SQL Queries** **20**
- 8.1 Basic SELECT Queries . . . . . 20
- 8.2 Joins . . . . . 20
- 8.3 Aggregate Functions . . . . . 20
- 8.4 GROUP BY and HAVING . . . . . 20
- 8.5 Subqueries . . . . . 21
- 8.6 Views . . . . . 21

**9 Transactions and Concurrency** **23**
9.1 Transaction Management . . . . . 23
9.2 COMMIT, ROLLBACK and SAVEPOINT . . . . . 23
9.3 Concurrency Control . . . . . 24

**10 PL/SQL Implementation** **25**
10.1 Functions . . . . . 25
10.2 Stored Procedures . . . . . 26
10.3 Cursors . . . . . 27
10.4 Triggers . . . . . 28

**11 File Organization and Indexing** **29**
11.1 File Organization . . . . . 29
11.2 ISAM . . . . . 29
11.3 B+ Tree Indexing . . . . . 29

**12 Testing** **30**
12.1 Test Cases . . . . . 30
12.2 Normal and Boundary Cases . . . . . 30
12.3 Invalid Input Testing . . . . . 31

**13 Reports** **31**
13.1 Booking Report . . . . . 31
13.2 Cancellation Report . . . . . 32
13.3 Revenue Report . . . . . 32
13.4 Overdue Rentals Report . . . . . 32

**14 Results and Discussion** **33**

**15 Conclusion** **34**

**16 SDG Mapping** **34**

**17 Future Scope** **35**

**18 Bibliography** **35**

# Abstract

The Sports Arena Management System is a database management system project designed to automate and streamline the operations of a sports tournament and the arena in which it is held. The system maintains complete records of tournaments, teams, players, venues, officials, matches, and scores. It replaces manual score sheets and scattered spreadsheets with a centralized relational database, thereby reducing data duplication, minimizing errors, and improving the speed and accuracy of fixture planning, score entry, and result publication.

The system allows organizers to register teams and players, generate fixtures, allocate venues and officials to matches, and enter scores as they happen, while the points table and player statistics are produced directly from the stored data. The database is designed using standard DBMS concepts, including entities, attributes, relationships, primary keys, foreign keys, and normalization, to ensure data consistency and integrity. The system can also generate useful reports such as fixture and venue booking details, cancelled or postponed matches, ticket revenue, and matches whose results are still pending.

This project demonstrates the practical application of DBMS concepts such as relational tables, constraints, SQL queries, joins, views, transactions, stored procedures, functions, cursors, triggers, and indexing. By automating the tournament process, the system reduces manual effort, improves accuracy, and provides a simple and efficient solution for sports tournament and arena management.



**Keywords:** Sports arena management, Database Management System, SQL, team registration, fixture generation, score entry, standings, player statistics, venues, officials, transactions.

# 1 Introduction

In today's fast-paced world, organized sports play an important role in promoting fitness, teamwork, and community engagement. Tournament organizers and sports arenas need to manage large amounts of information, including team registrations, player details, match fixtures, venue schedules, officials, live scores, and league standings. Managing these records manually can lead to problems such as data duplication, calculation errors, loss of information, clashing fixtures, and difficulty in tracking results and player performance.

The Sports Arena Management System is a database-based application developed to manage these operations in an organized manner. It allows organizers to register teams and players, generate fixtures, allocate venues and officials, enter scores, and publish standings and player statistics. By using a centralized database, the system reduces problems such as double-booked venues, incorrect points tables, and missing records. The project demonstrates important DBMS concepts such as ER modeling, normalization, constraints, SQL queries, transactions, PL/SQL functions and procedures, cursors, triggers, and indexing to provide a reliable and efficient tournament management system. The popularity of school, college, club, and professional leagues has grown rapidly, and an organizing body must handle many activities for every tournament, such as registering teams and verifying players, preparing the match schedule, assigning referees and scorers, booking the arena, recording every goal or point, and updating the league table after each match. When these activities are recorded manually or in separate spreadsheets, the information quickly becomes inconsistent, and problems such as clashing fixtures, wrong points tables, unrecorded results, and missing player statistics are common. The Sports Arena Management System addresses these issues by storing all data in a centralized relational database that is shared by the administrator, team managers, officials, and spectators. The database is organized into eight normalized tables, namely Tournament, Venue, Team, Player, Official, Match, Match_Official, and Score, and is linked through primary and foreign keys to ensure accuracy and integrity. Constraints, triggers, and stored procedures automatically check venue and team availability, validate score entries, generate fixtures, and update match status. The system also produces fixture, cancellation, revenue, and pending-result reports, together with the points table and player statistics, which support better decision-making. Overall, the project applies core DBMS concepts to build a reliable, efficient, and easy-to-maintain solution for modern sports tournament and arena management.

# 2 Problem Analysis

Tournament organizers often manage team details, player lists, match schedules, venue bookings, and scores manually or through separate records. This approach causes data duplication, errors in the points table, difficulty in checking venue and team availability, delayed score publication, and poor tracking of player performance. It also becomes difficult for the organizers to generate accurate reports on standings, cancelled fixtures, ticket revenue, and matches whose results have not been entered.

## 2.1 Problem Statement

Sports organizers need to manage a large amount of information, including team registrations, player details, match fixtures, venues, officials, scores, and results. When these records are maintained manually or in separate files, it becomes difficult to check venue availability, generate fair fixtures, record scores accurately, calculate standings, and avoid duplicate or missing data. This can lead to scheduling errors, delayed results, incorrect rankings, and poor decision-making. The proposed system provides a centralized database to manage these details efficiently and ensure accurate tournaments.

## 2.2 Users and Assumptions

The main users of the system are the administrator, team managers, officials, and spectators. The administrator manages tournaments, teams, players, venues, officials, fixtures, and scores. Team managers can register their teams and players and view their fixtures. Officials, such as referees and scorers, assist in running matches and entering scores, whereas spectators and the organizing committee use the reports to follow the standings, player statistics, and venue schedules.

The system makes the following assumptions:

- Every team provides valid details (including a coach and contact information) before it is registered in a tournament.
- Each player belongs to exactly one team and has a jersey number that is unique within that team.
- A venue can host only one match on a given date, and a team can play only one match per day.
- Each match is played between two different teams of the same tournament and is handled by one or more officials.
- A match may have many score entries, one for each goal or scoring event, so one match can have several scores.

### 2.3 Functional Requirements

The system provides functions for managing tournament, venue, team, player, official, match, and score information. It allows users to register teams and players, generate fixtures for a tournament, search for venues that are free on a given date, schedule and cancel matches, assign officials, and enter scores. The system prevents clashing fixtures at the same venue or for the same team, validates that scorers belong to the teams playing the match, and automatically computes match results, standings, and player statistics. It also supports generating useful reports such as fixture, cancellation, revenue, and pending-result reports.

## 3 ER Model

The Entity-Relationship (ER) model is used to represent the structure of the Sports Arena Management System and the relationships between its different entities. The main entities identified in the system are Tournament, Venue, Team, Player, Official, Match, and Score. Each entity contains attributes that describe its important information and is identified using a primary key. The many-to-many relationship between Match and Official is resolved with the associative entity Match_Official.

### 3.1 Entities and Attributes

The Tournament entity stores the competitions being organized, and Venue stores the arenas and grounds where matches are played. Team stores the registered teams, and Player stores the members of each team. Official stores the referees, scorers, and other match officials. Match records every fixture between two teams at a venue, Match_Official records which officials are assigned to which match, and Score records each scoring event (goal or point) made by a player in a match.

Table 1: Entities and Attributes of the System

| **Entity** | **Attributes** | **Keys** |
|---|---|---|
| Tournament | Tournament_ID, Tournament_Name, Sport, Start_Date, End_Date | PK: Tournament_ID |
| Venue | Venue_ID, Venue_Name, Address, City, Capacity, Phone | PK: Venue_ID |
| Team | Team_ID, Team_Name, Coach_Name, City, Email, Phone, Registration_Date, Tournament_ID | PK: Team_ID<br>FK: Tournament_ID |
| Player | Player_ID, First_Name, Last_Name, Date_Of_Birth, Playing_Position, Jersey_No, Phone, Email, Team_ID | PK: Player_ID<br>FK: Team_ID |
| Official | Official_ID, First_Name, Last_Name, Official_Role, Phone, Email, Experience_Years | PK: Official_ID |
| Match | Match_ID, Tournament_ID, Venue_ID, Home_Team_ID, Away_Team_ID, Match_Date, Round_Name, Match_Status, Ticket_Price, Tickets_Sold | PK: Match_ID<br>FK: Tournament_ID, Venue_ID, Home_Team_ID, Away_Team_ID |
| Match_Official | Match_ID, Official_ID, Duty_Role | PK: (Match_ID, Official_ID)<br>FK: Match_ID, Official_ID |
| Score | Score_ID, Match_ID, Player_ID, Score_Minute, Points_Scored | PK: Score_ID<br>FK: Match_ID, Player_ID |

## 3.2 Relationships

A tournament has many registered teams and many matches, and each team and match belongs to one tournament. A team has many players, and each player belongs to one team. A venue can host many matches, while each match is played at exactly one venue. A team can play many matches as the home team and many as the away team, and each match has exactly one home team and one away team. A match is handled by several officials, and an official can handle many matches. A match has many score entries, and each score entry is made by exactly one player, so a player can have many score entries.

## 3.3 Cardinality and Participation

Table 2: Relationships, Cardinality and Participation

| **Relationship** | **Entities** | **Cardinality** | **Participation** |
|---|---|---|---|
| Registers in | Tournament – Team | 1 : N | Team: total |
| Consists of | Team – Player | 1 : N | Player: total |
| Hosts | Venue – Match | 1 : N | Match: total |
| Includes | Tournament – Match | 1 : N | Match: total |
| Plays at home | Team – Match | 1 : N | Match: total |
| Plays away | Team – Match | 1 : N | Match: total |
| Officiates | Match – Official | M : N | Match: total, Official: partial |
| Records | Match – Score | 1 : N | Score: total |
| Scores | Player – Score | 1 : N | Score: total |

All relationships are one-to-many except Match–Official, which is many-to-many and is implemented through the Match_Official table. Every match must be associated with a tournament, a venue, a home team, and an away team, so its participation is mandatory. Entities such as Venue, Team, Player, and Official can exist independently before matches are scheduled.

### 3.4 ER Diagram

The ER diagram represents the entities, relationships, and cardinalities of the Sports Arena Management System. It provides a graphical representation of the database structure and serves as the basis for converting the ER model into relational tables.

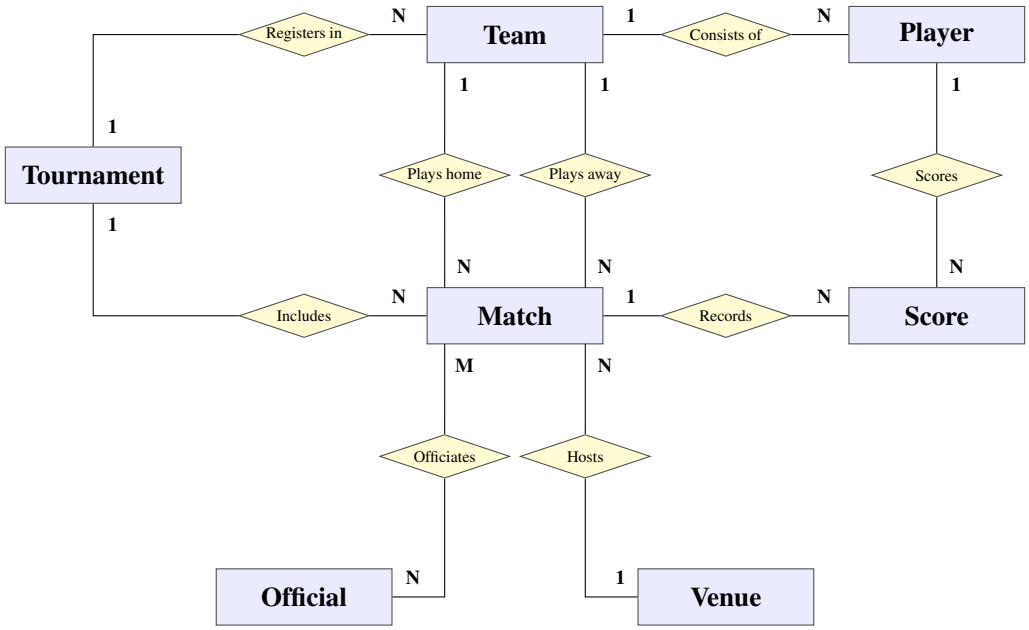



Figure 1: ER Diagram of Sports Arena Management System

# 4 Relational Model

The relational model represents the data of the Sports Arena Management System in the form of tables. Each table contains related attributes, with primary keys used to uniquely identify records and foreign keys used to establish relationships between tables.

### 4.1 Relational Schema

The main relational tables used in the system are Tournament, Venue, Team, Player, Official, Match, Match_Official, and Score. Primary keys are underlined.

- Tournament(Tournament_ID, Tournament_Name, Sport, Start_Date, End_Date)
- Venue(Venue_ID, Venue_Name, Address, City, Capacity, Phone)
- Team(Team_ID, Team_Name, Coach_Name, City, Email, Phone, Registration_Date, Tournament_ID)
- Player(Player_ID, First_Name, Last_Name, Date_Of_Birth, Playing_Position, Jersey_No, Phone, Email, Team_ID)

- Official(Official_ID, First_Name, Last_Name, Official_Role, Phone, Email, Experience_Years)
- Match(Match_ID, Tournament_ID, Venue_ID, Home_Team_ID, Away_Team_ID, Match_Date, Round_Name, Match_Status, Ticket_Price, Tickets_Sold)
- Match_Official(Match_ID, Official_ID, Duty_Role)
- Score(Score_ID, Match_ID, Player_ID, Score_Minute, Points_Scored)

### 4.2 Primary Keys and Foreign Keys

Each table has a primary key that uniquely identifies every record. Foreign keys are used to connect related tables. In addition, Team_Name and the Email columns are declared UNIQUE, the pair (Team_ID, Jersey_No) is UNIQUE in Player so that no two players of a team share a jersey number, and Match_Official uses the pair (Match_ID, Official_ID) as its primary key so that an official is assigned to a match only once.

Table 3: Foreign Keys in the Schema

| **Table** | **Foreign Key** | **References** |
|---|---|---|
| Team | Tournament_ID | Tournament(Tournament_ID) |
| Player | Team_ID | Team(Team_ID) |
| Match | Tournament_ID | Tournament(Tournament_ID) |
| Match | Venue_ID | Venue(Venue_ID) |
| Match | Home_Team_ID | Team(Team_ID) |
| Match | Away_Team_ID | Team(Team_ID) |
| Match_Official | Match_ID | Match(Match_ID) |
| Match_Official | Official_ID | Official(Official_ID) |
| Score | Match_ID | Match(Match_ID) |
| Score | Player_ID | Player(Player_ID) |

## 5 Normalization

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies. The Sports Arena Management System is normalized up to Third Normal Form (3NF), and every table also satisfies BCNF, to ensure that the data is stored efficiently and consistently. The design starts from a single unnormalized table, such as the one a manual score book would produce, which stores tournament, venue, team, player, official, match, and score details together.

## 5.1 Functional Dependencies

Functional dependencies describe how one attribute (or set of attributes) uniquely determines others. In the proposed system, the following functional dependencies are identified:

- Tournament_ID → Tournament_Name, Sport, Start_Date, End_Date
- Venue_ID → Venue_Name, Address, City, Capacity, Phone
- Team_ID → Team_Name, Coach_Name, City, Email, Phone, Registration_Date, Tournament_ID
- Player_ID → First_Name, Last_Name, Date_Of_Birth, Playing_Position, Jersey_No, Phone, Email, Team_ID
- Official_ID → First_Name, Last_Name, Official_Role, Phone, Email, Experience_-Years
- Match_ID → Tournament_ID, Venue_ID, Home_Team_ID, Away_Team_ID, Match_-Date, Round_Name, Match_Status, Ticket_Price, Tickets_Sold
- (Match_ID, Official_ID) → Duty_Role
- Score_ID → Match_ID, Player_ID, Score_Minute, Points_Scored
- Team_Name → Team_ID, Email → Player_ID, Email → Official_ID, (Team_ID, Jersey_No) → Player_ID (candidate keys)

These dependencies ensure that each non-key attribute depends on the appropriate key of its relation, and every determinant is a candidate key.

## 5.2 First Normal Form (1NF)

A relation is in First Normal Form when all attributes contain atomic values, there are no repeating groups, and each row is uniquely identified by a primary key. The initial table violates 1NF because a match may have several officials in one field, a match may have several scorers in one row, and a team may have several players in one row.

Table 4: Before 1NF (multi-valued cells)

| **match_id** | **venue_name** | **teams** | **officials** | **scorers** |
|---|---|---|---|---|
| M101 | Arena Stadium A | Falcons FC vs Titans FC | R. Sharma (Referee), K. Rao (Scorer) | Arjun Mehta 23', Imran Khan 55' |

To achieve 1NF, multi-valued attributes are split into single values, each scoring event is stored in its own row, officials are moved to a separate table, and primary keys are defined.

Table 5: Match_Score Table in First Normal Form (1NF), PK = (match_id, score_id)

| match_id | score_id | venue_name | home_team | away_team | player_name | minute | points |
|---|---|---|---|---|---|---|---|
| M101 | S1 | Arena Stadium A | Falcons FC | Titans FC | Arjun Mehta | 23 | 1 |
| M101 | S2 | Arena Stadium A | Falcons FC | Titans FC | Imran Khan | 55 | 1 |

Official assignments are stored as Match_Official(match_id, official_id, duty_role). Every cell now holds one atomic value, but venue and team details are still repeated on each scoring row. This is removed in 2NF.

## 5.3 Second Normal Form (2NF)

A relation is in Second Normal Form when it is already in 1NF and every non-key attribute is fully dependent on the complete primary key. Partial dependency should not exist. In the 1NF table the key is (match_id, score_id), but venue and team details depend only on match_id, while the scorer details depend only on score_id. These are partial dependencies.

The table is therefore split into a match table and a score table, each with a single attribute key.

Table 6: Match_Venue Table in 2NF, PK = match_id

| match_id | home_id | home_team | away_id | away_team | venue_id | venue_name | capacity | match_date |
|---|---|---|---|---|---|---|---|---|
| M101 | T01 | Falcons FC | T02 | Titans FC | V01 | Arena Stadium A | 5000 | 2026-10-03 |
| M102 | T03 | Warriors Utd | T04 | Royals SC | V02 | Lakeside Complex | 3000 | 2026-10-04 |
| M103 | T01 | Falcons FC | T03 | Warriors Utd | V01 | Arena Stadium A | 5000 | 2026-10-10 |

Table 7: Score Table in 2NF, PK = score_id, FK = match_id

| score_id | match_id | player_name | minute | points |
|---|---|---|---|---|
| S1 | M101 | Arjun Mehta | 23 | 1 |
| S2 | M101 | Imran Khan | 55 | 1 |

No partial dependencies remain, but the Match_Venue table still repeats team, venue, and capacity details for every match, which are transitive dependencies.

## 5.4 Third Normal Form (3NF)

A relation is in Third Normal Form when it is in 2NF and there is no transitive dependency between non-key attributes. In the Match_Venue table the following transitive dependencies exist:

- match_id $\rightarrow$ venue_id $\rightarrow$ venue name, city, capacity

- match_id → home_team_id → home team name, coach, city
- match_id → away_team_id → away team name, coach, city
- match_id → tournament_id → tournament name, sport
- score_id → player_id → player name, jersey number, team

These cause update anomalies (changing the capacity of a venue means editing many rows), insertion anomalies (a new team or venue cannot be stored until it plays a match), and deletion anomalies (deleting the last match at a venue erases the venue or team details). Each determinant is therefore moved into its own table and kept in the original table only as a foreign key. Scoring details are placed in a separate Score table because they exist only after a match has been played.

Table 8: Team Table (3NF), PK = team_id

| **team_id** | **team_name** | **coach_name** | **city** |
|---|---|---|---|
| T01 | Falcons FC | Rakesh Varma | Hyderabad |
| T02 | Titans FC | Suresh Iyer | Warangal |

Table 9: Venue Table (3NF), PK = venue_id

| **venue_id** | **venue_name** | **city** | **capacity** |
|---|---|---|---|
| V01 | Arena Stadium A | Hyderabad | 5000 |
| V02 | Lakeside Sports Complex | Hyderabad | 3000 |

Table 10: Player Table (3NF), PK = player_id, FK = team_id

| **player_id** | **first_name** | **last_name** | **jersey_no** | **team_id** |
|---|---|---|---|---|
| P01 | Arjun | Mehta | 9 | T01 |
| P02 | Imran | Khan | 10 | T02 |

Table 11: Match Table (3NF), PK = match_id, FK = tournament_id, venue_id, home_team_id, away_team_id

| **match_id** | **tournament_id** | **venue_id** | **home_team_id** | **away_team_id** | **match_date** |
|---|---|---|---|---|---|
| M101 | TR1 | V01 | T01 | T02 | 2026-10-03 |
| M102 | TR1 | V02 | T03 | T04 | 2026-10-04 |
| M103 | TR1 | V01 | T01 | T03 | 2026-10-10 |

Thus, the final relations are separated as shown below.

Table 12: Final 3NF Relations

| No. | Relation | Primary Key | Foreign Keys |
|---|---|---|---|
| 1 | Tournament | Tournament_ID | – |
| 2 | Venue | Venue_ID | – |
| 3 | Team | Team_ID | Tournament_ID |
| 4 | Player | Player_ID | Team_ID |
| 5 | Official | Official_ID | – |
| 6 | Match | Match_ID | Tournament_ID, Venue_ID, Home_-Team_ID, Away_Team_ID |
| 7 | Match_Official | (Match_ID, Official_ID) | Match_ID, Official_ID |
| 8 | Score | Score_ID | Match_ID, Player_ID |

In every relation, each determinant is a candidate key, so the database is also in Boyce-Codd Normal Form (BCNF). Normalization reduces data redundancy and improves data consistency and maintainability.

# 6 SQL DDL Commands

Data Definition Language (DDL) commands are used to create and modify the structure of database tables. In the Sports Arena Management System, DDL commands are used to create tables and define constraints such as primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT. The statements below use Oracle SQL syntax.

## 6.1 Table Creation

The tables are created in the order of dependency, so that parent tables exist before the tables that reference them.

```
CREATE TABLE Tournament (
  Tournament_ID NUMBER PRIMARY KEY,
  Tournament_Name VARCHAR2(100) NOT NULL,
  Sport VARCHAR2(50) NOT NULL,
  Start_Date DATE,
  End_Date DATE,
  CHECK (End_Date >= Start_Date)
);
```

```
CREATE TABLE Venue (
  Venue_ID NUMBER PRIMARY KEY,
  Venue_Name VARCHAR2(100) NOT NULL,
  Address VARCHAR2(200),
  City VARCHAR2(50),
  Capacity NUMBER CHECK (Capacity > 0),
  Phone VARCHAR2(15)
```

```
);
```

```
CREATE TABLE Team (
  Team_ID NUMBER PRIMARY KEY,
  Team_Name VARCHAR2(100) UNIQUE NOT NULL,
  Coach_Name VARCHAR2(100),
  City VARCHAR2(50),
  Email VARCHAR2(100) UNIQUE,
  Phone VARCHAR2(15),
  Registration_Date DATE DEFAULT SYSDATE,
  Tournament_ID NUMBER,
  FOREIGN KEY (Tournament_ID) REFERENCES Tournament(Tournament_ID)
);
```

```
CREATE TABLE Player (
  Player_ID NUMBER PRIMARY KEY,
  First_Name VARCHAR2(50) NOT NULL,
  Last_Name VARCHAR2(50),
  Date_Of_Birth DATE,
  Playing_Position VARCHAR2(30),
  Jersey_No NUMBER(2) CHECK (Jersey_No BETWEEN 0 AND 99),
  Phone VARCHAR2(15),
  Email VARCHAR2(100) UNIQUE,
  Team_ID NUMBER NOT NULL,
  UNIQUE (Team_ID, Jersey_No),
  FOREIGN KEY (Team_ID) REFERENCES Team(Team_ID)
);
```

```
CREATE TABLE Official (
  Official_ID NUMBER PRIMARY KEY,
  First_Name VARCHAR2(50) NOT NULL,
  Last_Name VARCHAR2(50),
  Official_Role VARCHAR2(30),
  Phone VARCHAR2(15),
  Email VARCHAR2(100) UNIQUE,
  Experience_Years NUMBER(2) CHECK (Experience_Years >= 0)
);
```

```
CREATE TABLE Match (
  Match_ID NUMBER PRIMARY KEY,
  Tournament_ID NUMBER NOT NULL,
  Venue_ID NUMBER NOT NULL,
  Home_Team_ID NUMBER NOT NULL,
  Away_Team_ID NUMBER NOT NULL,
  Match_Date DATE NOT NULL,
  Round_Name VARCHAR2(30),
  Match_Status VARCHAR2(20) DEFAULT 'Scheduled'
    CHECK (Match_Status IN
      ('Scheduled','Completed','Cancelled','Postponed')),
  Ticket_Price NUMBER(8,2) CHECK (Ticket_Price >= 0),
  Tickets_Sold NUMBER DEFAULT 0 CHECK (Tickets_Sold >= 0),
  CHECK (Home_Team_ID <> Away_Team_ID),
  FOREIGN KEY (Tournament_ID) REFERENCES Tournament(Tournament_ID),
  FOREIGN KEY (Venue_ID) REFERENCES Venue(Venue_ID),
  FOREIGN KEY (Home_Team_ID) REFERENCES Team(Team_ID),
  FOREIGN KEY (Away_Team_ID) REFERENCES Team(Team_ID)
);
```

```
CREATE TABLE Match_Official (
  Match_ID NUMBER,
  Official_ID NUMBER,
  Duty_Role VARCHAR2(30)
```

```
    CHECK (Duty_Role IN
      ('Main Referee','Assistant Referee','Scorer','Timekeeper')),
  PRIMARY KEY (Match_ID, Official_ID),
  FOREIGN KEY (Match_ID) REFERENCES Match(Match_ID),
  FOREIGN KEY (Official_ID) REFERENCES Official(Official_ID)
);
```

```
CREATE TABLE Score (
  Score_ID NUMBER PRIMARY KEY,
  Match_ID NUMBER NOT NULL,
  Player_ID NUMBER NOT NULL,
  Score_Minute NUMBER(3) CHECK (Score_Minute BETWEEN 0 AND 120),
  Points_Scored NUMBER(3) DEFAULT 1 CHECK (Points_Scored > 0),
  FOREIGN KEY (Match_ID) REFERENCES Match(Match_ID),
  FOREIGN KEY (Player_ID) REFERENCES Player(Player_ID)
);
```

## 6.2 Constraints

Primary keys uniquely identify records, while foreign keys establish relationships between tables. The NOT NULL constraint ensures that required fields contain values. UNIQUE prevents duplicate values such as repeated team names, emails, or two players of the same team wearing the same jersey number. CHECK restricts values according to specified conditions, for example positive capacities, valid match status values, a match whose home and away teams are different, a tournament end date that is not earlier than its start date, and a scoring minute between 0 and 120. DEFAULT supplies a value when none is given, such as the initial match status.

## 6.3 ALTER Operation

The ALTER command is used to modify an existing table.

```
ALTER TABLE Venue ADD Parking_Capacity NUMBER(6);
ALTER TABLE Player MODIFY Phone VARCHAR2(20);
```

## 6.4 DROP Operation

The DROP command is used to remove a table and its stored records.

```
DROP TABLE Match_Official CASCADE CONSTRAINTS;
```

# 7 SQL DML Commands

Data Manipulation Language (DML) commands are used to insert, modify, and delete records in database tables. In the Sports Arena Management System, DML commands are used to manage tournament, team, player, venue, match, and score information.

## 7.1 INSERT

The INSERT command is used to add new records to a table. The following examples insert sample records.

```
INSERT INTO Tournament VALUES (1, 'Arena Premier Cup 2026', 'Football',
DATE '2026-10-01', DATE '2026-10-31');
```

```
INSERT INTO Venue VALUES (1, 'Arena Stadium A', 'Gachibowli', 'Hyderabad', 5000, '04011112222');
INSERT INTO Venue VALUES (2, 'Lakeside Sports Complex', 'Secunderabad Road', 'Hyderabad', 3000,
    '04033334444');
INSERT INTO Venue VALUES (3, 'City Indoor Arena', 'Kukatpally', 'Hyderabad', 1500, '04055556666');
```

```
INSERT INTO Team VALUES (101, 'Falcons FC', 'Rakesh Varma', 'Hyderabad',
'falcons@arena.com', '9000010001', DATE '2026-09-15', 1);
INSERT INTO Team VALUES (102, 'Titans FC', 'Suresh Iyer', 'Warangal',
'titans@arena.com', '9000010002', DATE '2026-09-16', 1);
INSERT INTO Team VALUES (103, 'Warriors United', 'Anita Sharma', 'Secunderabad',
'warriors@arena.com', '9000010003', DATE '2026-09-17', 1);
INSERT INTO Team VALUES (104, 'Royals SC', 'Mohan Das', 'Vijayawada',
'royals@arena.com', '9000010004', DATE '2026-09-18', 1);
```

```
INSERT INTO Player VALUES (201, 'Arjun', 'Mehta', DATE '1999-03-12', 'Forward', 9,
'9100000001', 'arjun@arena.com', 101);
INSERT INTO Player VALUES (202, 'Kabir', 'Nair', DATE '1999-07-21', 'Midfielder', 8,
'9100000002', 'kabir@arena.com', 101);
INSERT INTO Player VALUES (203, 'Imran', 'Khan', DATE '1998-01-30', 'Forward', 10,
'9100000003', 'imran@arena.com', 102);
INSERT INTO Player VALUES (204, 'Rahul', 'Verma', DATE '2001-05-19', 'Defender', 4,
'9100000004', 'rahul@arena.com', 102);
INSERT INTO Player VALUES (205, 'Vikram', 'Singh', DATE '1997-09-09', 'Forward', 11,
'9100000005', 'vikram@arena.com', 103);
INSERT INTO Player VALUES (206, 'Rohan', 'Das', DATE '2000-12-02', 'Midfielder', 6,
'9100000006', 'rohan@arena.com', 103);
INSERT INTO Player VALUES (207, 'Dev', 'Patel', DATE '1999-04-25', 'Forward', 7,
'9100000007', 'dev@arena.com', 104);
INSERT INTO Player VALUES (208, 'Sameer', 'Ali', DATE '2002-08-17', 'Goalkeeper', 1,
'9100000008', 'sameer@arena.com', 104);
```

```
INSERT INTO Official VALUES (301, 'Ramesh', 'Sharma', 'Referee', '9200000001',
'ramesh@arena.com', 8);
INSERT INTO Official VALUES (302, 'Kiran', 'Rao', 'Scorer', '9200000002',
'kiran@arena.com', 5);
INSERT INTO Official VALUES (303, 'Neha', 'Joshi', 'Assistant Referee', '9200000003',
'neha@arena.com', 6);
```

```
INSERT INTO Match VALUES (401, 1, 1, 101, 102, DATE '2026-10-03', 'Round 1',
'Completed', 250, 4200);
INSERT INTO Match VALUES (402, 1, 2, 103, 104, DATE '2026-10-04', 'Round 1',
'Completed', 150, 2400);
INSERT INTO Match VALUES (403, 1, 1, 101, 103, DATE '2026-10-10', 'Round 2',
'Completed', 250, 4650);
```

```
INSERT INTO Match VALUES (404, 1, 2, 102, 104, DATE '2026-10-09', 'Round 2',
'Scheduled', 150, 1800);
INSERT INTO Match VALUES (405, 1, 1, 101, 104, DATE '2026-10-18', 'Round 3',
'Scheduled', 250, 0);
INSERT INTO Match VALUES (406, 1, 3, 102, 103, DATE '2026-10-20', 'Round 3',
'Scheduled', 200, 0);
```

```
INSERT INTO Match_Official VALUES (401, 301, 'Main Referee');
INSERT INTO Match_Official VALUES (401, 303, 'Assistant Referee');
INSERT INTO Match_Official VALUES (401, 302, 'Scorer');
INSERT INTO Match_Official VALUES (402, 301, 'Main Referee');
INSERT INTO Match_Official VALUES (402, 302, 'Scorer');
INSERT INTO Match_Official VALUES (403, 303, 'Main Referee');
INSERT INTO Match_Official VALUES (403, 302, 'Scorer');
INSERT INTO Match_Official VALUES (405, 301, 'Main Referee');
```

```
INSERT INTO Score VALUES (501, 401, 201, 23, 1);
INSERT INTO Score VALUES (502, 401, 203, 55, 1);
INSERT INTO Score VALUES (503, 401, 201, 67, 1);
INSERT INTO Score VALUES (504, 402, 205, 30, 1);
INSERT INTO Score VALUES (505, 402, 207, 75, 1);
INSERT INTO Score VALUES (506, 403, 205, 12, 1);
INSERT INTO Score VALUES (507, 403, 202, 40, 1);
INSERT INTO Score VALUES (508, 403, 206, 60, 1);
COMMIT;
```

## 7.2 UPDATE

The UPDATE command is used to modify existing records in a table. For example, the status of a match can be changed when it is postponed or cancelled, and the ticket price of a round can be revised.

```
UPDATE Match SET Match_Status = 'Postponed' WHERE Match_ID = 405;
UPDATE Match SET Match_Status = 'Cancelled' WHERE Match_ID = 406;
UPDATE Match SET Ticket_Price = 300 WHERE Match_ID = 405;
```

## 7.3 DELETE

The DELETE command is used to remove records from a table. For example, the official assignments of a postponed match can be removed until the match is rescheduled, and a wrongly entered score can be deleted.

```
DELETE FROM Match_Official WHERE Match_ID = 405;
DELETE FROM Score WHERE Score_ID = 999;
```

Records that are referenced by other tables (for example, a team with players or a match with scores) cannot be deleted because of the foreign key constraints, which protects data integrity.

# 8 SQL Queries

SQL queries are used to retrieve and analyze data stored in the Sports Arena Management System. The system uses SELECT statements, joins, aggregate functions, subqueries, and views to obtain useful information such as fixtures, standings, and player statistics.

## 8.1 Basic SELECT Queries

```
SELECT * FROM Team;
SELECT First_Name, Last_Name FROM Player WHERE Playing_Position = 'Forward';
SELECT * FROM Match WHERE Match_Status = 'Scheduled';
SELECT * FROM Venue WHERE Venue_ID = 1;
```

## 8.2 Joins

Joins are used to retrieve related information from two or more tables.

```
-- Players with their team and jersey number
SELECT P.First_Name, P.Last_Name, P.Jersey_No, T.Team_Name
FROM Player P JOIN Team T ON P.Team_ID = T.Team_ID;
```

```
-- Fixtures with team names and venue
SELECT M.Match_ID, H.Team_Name AS Home_Team, A.Team_Name AS Away_Team,
       V.Venue_Name, M.Match_Date
FROM Match M
JOIN Team H ON M.Home_Team_ID = H.Team_ID
JOIN Team A ON M.Away_Team_ID = A.Team_ID
JOIN Venue V ON M.Venue_ID = V.Venue_ID;
```

## 8.3 Aggregate Functions

Aggregate functions such as COUNT, SUM, AVG, MAX, and MIN are used to perform calculations on records.

```
SELECT COUNT(*) AS Total_Matches FROM Match;
SELECT SUM(Points_Scored) AS Total_Goals FROM Score;
SELECT AVG(Capacity) AS Average_Capacity FROM Venue;
SELECT MAX(Capacity) AS Largest_Venue FROM Venue;
```

## 8.4 GROUP BY and HAVING

GROUP BY is used to group records, while HAVING is used to apply conditions to groups.

```
SELECT Match_Status, COUNT(*) AS Total
FROM Match GROUP BY Match_Status;
```

```
SELECT Player_ID, SUM(Points_Scored) AS Total_Goals
FROM Score GROUP BY Player_ID HAVING SUM(Points_Scored) > 1;
```

## 8.5 Subqueries

A subquery is a query written inside another query. The first query displays venues whose capacity is above the average capacity. The second finds venues that are free on a requested date (10 October 2026), which is the core availability check of the system.

```
SELECT Venue_Name FROM Venue
WHERE Capacity > (SELECT AVG(Capacity) FROM Venue);
```

```
SELECT Venue_ID, Venue_Name FROM Venue
WHERE Venue_ID NOT IN (SELECT Venue_ID FROM Match
                       WHERE Match_Status IN ('Scheduled','Completed')
                       AND Match_Date = DATE '2026-10-10');
```

## 8.6 Views

A view is a virtual table created from the result of an SQL query. The Match_Details view simplifies fixture look-ups. The other views derive the results, the points table, and the player statistics directly from the stored scores, so the standings never have to be updated by hand. A win earns 3 points, a draw 1 point, and a loss 0 points (an assumed business rule).

```
CREATE VIEW Match_Details AS
SELECT M.Match_ID, H.Team_Name AS Home_Team, A.Team_Name AS Away_Team,
       V.Venue_Name, M.Match_Date, M.Match_Status
FROM Match M
JOIN Team H ON M.Home_Team_ID = H.Team_ID
JOIN Team A ON M.Away_Team_ID = A.Team_ID
JOIN Venue V ON M.Venue_ID = V.Venue_ID;
```

```
CREATE VIEW Match_Result AS
SELECT M.Match_ID, M.Home_Team_ID, M.Away_Team_ID,
  (SELECT COALESCE(SUM(S.Points_Scored), 0)
   FROM Score S JOIN Player P ON S.Player_ID = P.Player_ID
   WHERE S.Match_ID = M.Match_ID AND P.Team_ID = M.Home_Team_ID) AS Home_Score,
  (SELECT COALESCE(SUM(S.Points_Scored), 0)
   FROM Score S JOIN Player P ON S.Player_ID = P.Player_ID
   WHERE S.Match_ID = M.Match_ID AND P.Team_ID = M.Away_Team_ID) AS Away_Score
FROM Match M WHERE M.Match_Status = 'Completed';
```

```
CREATE VIEW Team_Match_Result AS
SELECT Match_ID, Home_Team_ID AS Team_ID,
       Home_Score AS Goals_For, Away_Score AS Goals_Against FROM Match_Result
UNION ALL
SELECT Match_ID, Away_Team_ID, Away_Score, Home_Score FROM Match_Result;
```

```
CREATE VIEW Team_Standings AS
SELECT T.Team_ID, T.Team_Name, COUNT(R.Match_ID) AS Played,
  SUM(CASE WHEN R.Goals_For > R.Goals_Against THEN 1 ELSE 0 END) AS Won,
  SUM(CASE WHEN R.Goals_For = R.Goals_Against THEN 1 ELSE 0 END) AS Drawn,
  SUM(CASE WHEN R.Goals_For < R.Goals_Against THEN 1 ELSE 0 END) AS Lost,
  COALESCE(SUM(R.Goals_For), 0) AS Goals_For,
  COALESCE(SUM(R.Goals_Against), 0) AS Goals_Against,
  SUM(CASE WHEN R.Goals_For > R.Goals_Against THEN 3
           WHEN R.Goals_For = R.Goals_Against THEN 1 ELSE 0 END) AS Points
FROM Team T LEFT JOIN Team_Match_Result R ON T.Team_ID = R.Team_ID
GROUP BY T.Team_ID, T.Team_Name;
```

```
CREATE VIEW Player_Statistics AS
SELECT P.Player_ID, P.First_Name || ' ' || P.Last_Name AS Player_Name,
       T.Team_Name, COUNT(S.Score_ID) AS Scoring_Events,
       COALESCE(SUM(S.Points_Scored), 0) AS Total_Points
FROM Player P JOIN Team T ON P.Team_ID = T.Team_ID
LEFT JOIN Score S ON P.Player_ID = S.Player_ID
GROUP BY P.Player_ID, P.First_Name, P.Last_Name, T.Team_Name;
```

The views can be accessed using:

```
SELECT * FROM Match_Details;
```

```
SELECT Team_Name, Played, Won, Drawn, Lost, Goals_For, Goals_Against,
       Goals_For - Goals_Against AS Goal_Diff, Points
FROM Team_Standings
ORDER BY Points DESC, Goals_For - Goals_Against DESC, Goals_For DESC;
```

```
SELECT Player_Name, Team_Name, Scoring_Events, Total_Points
FROM Player_Statistics ORDER BY Total_Points DESC, Player_Name;
```

The sample points table and player statistics produced from the data inserted earlier are shown below.

Table 13: Sample Team Standings (Team_Standings view)

| Pos | Team | P | W | D | L | GF | GA | GD | Pts |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Warriors United | 2 | 1 | 1 | 0 | 3 | 2 | +1 | 4 |
| 2 | Falcons FC | 2 | 1 | 0 | 1 | 3 | 3 | 0 | 3 |
| 3 | Royals SC | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 4 | Titans FC | 1 | 0 | 0 | 1 | 1 | 2 | -1 | 0 |

Table 14: Sample Player Statistics (Player_Statistics view)

| Player | Team | Scoring Events | Total Points |
|---|---|---|---|
| Arjun Mehta | Falcons FC | 2 | 2 |
| Vikram Singh | Warriors United | 2 | 2 |
| Dev Patel | Royals SC | 1 | 1 |
| Imran Khan | Titans FC | 1 | 1 |
| Kabir Nair | Falcons FC | 1 | 1 |
| Rohan Das | Warriors United | 1 | 1 |
| Rahul Verma | Titans FC | 0 | 0 |
| Sameer Ali | Royals SC | 0 | 0 |

# 9 Transactions and Concurrency

Transactions are a sequence of database operations performed as a single logical unit. In the Sports Arena Management System, transactions help ensure that fixture scheduling, score entry, and cancellation operations are completed correctly and consistently.

## 9.1 Transaction Management

A transaction begins when a database operation is performed and ends when the changes are either permanently saved or cancelled. For example, finishing a match involves entering every scoring event, updating the ticket sales, and changing the match status to Completed. Either all of these steps succeed or none of them do, so that the points table never reflects a half-entered result.

## 9.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT permanently saves the changes made by a transaction. ROLLBACK cancels the changes made since the beginning of the transaction or since the last SAVEPOINT. SAVEPOINT creates a point within a transaction to which the transaction can be rolled back.

```
INSERT INTO Match VALUES (407, 1, 3, 101, 102, DATE '2026-10-25', 'Exhibition',
'Scheduled', 200, 0);
COMMIT;
```

```
UPDATE Match SET Match_Status = 'Cancelled' WHERE Match_ID = 407;
ROLLBACK;
```

A SAVEPOINT can be used when multiple operations are performed in a transaction.

```
SAVEPOINT score_save;
INSERT INTO Score VALUES (509, 403, 202, 85, 1);
ROLLBACK TO score_save;
```

## 9.3 Concurrency Control

Concurrency control manages simultaneous transactions accessing the same database. In a sports arena system, it prevents two organizers from booking the same venue for the same date at the same time, and prevents two scorers from corrupting the score of the same match. Proper concurrency control maintains data consistency and prevents problems such as lost updates and clashing fixtures.

For example, a transaction can lock a venue record before confirming a fixture so that another transaction cannot book the same venue until the first one finishes. Similarly, the match row can be locked while its scores are being entered.

```
SELECT * FROM Venue WHERE Venue_ID = 1 FOR UPDATE;
SELECT * FROM Match WHERE Match_ID = 403 FOR UPDATE;
```

Thus, transaction management and concurrency control ensure reliable and consistent tournaments.

# 10 PL/SQL Implementation

PL/SQL is a procedural extension of SQL that allows programming constructs such as variables, conditions, loops, procedures, functions, cursors, and triggers. In the Sports Arena Management System, PL/SQL is used to implement database operations and automate tasks such as availability checking, fixture generation, score validation, and points calculation.

## 10.1 Functions

A function is a stored PL/SQL program that accepts parameters and returns a value. The first function returns the total points (goals) scored by a player. The second calculates the league points earned by a team, using the results derived from the stored scores.

```
CREATE OR REPLACE FUNCTION Get_Player_Points(P_ID NUMBER)
RETURN NUMBER IS
  V_Total NUMBER;
BEGIN
  SELECT COALESCE(SUM(Points_Scored), 0) INTO V_Total FROM Score
  WHERE Player_ID = P_ID;
  RETURN V_Total;
END;
/
```

```
CREATE OR REPLACE FUNCTION Get_Team_Points(P_Team_ID NUMBER)
RETURN NUMBER IS
  V_Points NUMBER;
BEGIN
  SELECT COALESCE(SUM(CASE WHEN Goals_For > Goals_Against THEN 3
                           WHEN Goals_For = Goals_Against THEN 1
                           ELSE 0 END), 0)
  INTO V_Points FROM Team_Match_Result
  WHERE Team_ID = P_Team_ID;
  RETURN V_Points;
END;
/
```

The functions can be called using:

```
SELECT Get_Player_Points(201) FROM DUAL;
SELECT Get_Team_Points(103) FROM DUAL;
```

## 10.2 Stored Procedures

A stored procedure is a named PL/SQL program used to perform a specific operation. The procedure Schedule_Match rejects a fixture if the venue or either team already has a match on the requested date; Cancel_Match cancels a match that has not yet been completed; Record_Score enters a scoring event for a match; and Complete_Match marks a match as completed once its date has arrived, after which it appears in the standings.

```
CREATE OR REPLACE PROCEDURE Schedule_Match(
  P_Match_ID NUMBER, P_Tournament_ID NUMBER, P_Venue_ID NUMBER,
  P_Home_ID NUMBER, P_Away_ID NUMBER, P_Date DATE,
  P_Round VARCHAR2, P_Price NUMBER) AS
  V_Count NUMBER;
BEGIN
  SELECT COUNT(*) INTO V_Count FROM Match
  WHERE Match_Date = P_Date
  AND Match_Status IN ('Scheduled','Completed')
  AND (Venue_ID = P_Venue_ID
       OR Home_Team_ID IN (P_Home_ID, P_Away_ID)
       OR Away_Team_ID IN (P_Home_ID, P_Away_ID));
  IF V_Count > 0 THEN
    RAISE_APPLICATION_ERROR(-20001,
      'Venue or team is already booked for the selected date');
  END IF;
  INSERT INTO Match VALUES (P_Match_ID, P_Tournament_ID, P_Venue_ID,
    P_Home_ID, P_Away_ID, P_Date, P_Round, 'Scheduled', P_Price, 0);
  COMMIT;
END;
/
```

```
CREATE OR REPLACE PROCEDURE Cancel_Match(M_ID NUMBER) AS
BEGIN
  UPDATE Match SET Match_Status = 'Cancelled'
  WHERE Match_ID = M_ID AND Match_Status <> 'Completed';
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20002,
      'Match not found or already completed');
  END IF;
  COMMIT;
END;
/
```

```
CREATE OR REPLACE PROCEDURE Record_Score(
  P_Score_ID NUMBER, P_Match_ID NUMBER, P_Player_ID NUMBER,
  P_Minute NUMBER, P_Points NUMBER) AS
BEGIN
  INSERT INTO Score VALUES (P_Score_ID, P_Match_ID, P_Player_ID,
    P_Minute, P_Points);
  COMMIT;
END;
/
```

```
CREATE OR REPLACE PROCEDURE Complete_Match(P_Match_ID NUMBER) AS
  V_Date DATE;
BEGIN
  SELECT Match_Date INTO V_Date FROM Match
  WHERE Match_ID = P_Match_ID AND Match_Status = 'Scheduled';
  IF V_Date > TRUNC(SYSDATE) THEN
    RAISE_APPLICATION_ERROR(-20004, 'Match has not been played yet');
```

```
  END IF;
  UPDATE Match SET Match_Status = 'Completed'
  WHERE Match_ID = P_Match_ID;
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20005, 'Match is not in Scheduled status');
END;
/
```

The procedures can be executed using:

```
EXEC Schedule_Match(407, 1, 3, 101, 102, DATE '2026-10-25', 'Exhibition', 200);
EXEC Record_Score(509, 404, 203, 41, 1);
EXEC Complete_Match(404);
EXEC Cancel_Match(406);
```

## 10.3 Cursors

A cursor is used to process the rows returned by a query one at a time. The following cursor displays the details of all teams registered in a tournament.

```
DECLARE
  CURSOR Team_Cursor IS
    SELECT Team_ID, Team_Name, Coach_Name FROM Team
    WHERE Tournament_ID = 1 ORDER BY Team_ID;
BEGIN
  FOR T IN Team_Cursor LOOP
    DBMS_OUTPUT.PUT_LINE(T.Team_ID || ' ' || T.Team_Name || ' - ' || T.Coach_Name);
  END LOOP;
END;
/
```

Fixture generation also uses a cursor. The procedure Generate_Fixtures loops over the teams registered in a tournament and creates a single round-robin schedule in which every team plays every other team once, with one match per day at the chosen venue starting from the given date.

```
CREATE OR REPLACE PROCEDURE Generate_Fixtures(
  P_Tournament_ID NUMBER, P_Start DATE, P_Venue_ID NUMBER) AS
  CURSOR Team_Cur IS
    SELECT Team_ID FROM Team
    WHERE Tournament_ID = P_Tournament_ID ORDER BY Team_ID;
  V_Match_ID NUMBER;
  V_Day NUMBER := 0;
BEGIN
  SELECT COALESCE(MAX(Match_ID), 0) INTO V_Match_ID FROM Match;
  FOR H IN Team_Cur LOOP
    FOR A IN (SELECT Team_ID FROM Team
              WHERE Tournament_ID = P_Tournament_ID
              AND Team_ID > H.Team_ID ORDER BY Team_ID) LOOP
      V_Match_ID := V_Match_ID + 1;
      INSERT INTO Match VALUES (V_Match_ID, P_Tournament_ID, P_Venue_ID,
        H.Team_ID, A.Team_ID, P_Start + V_Day, 'League',
        'Scheduled', 0, 0);
      V_Day := V_Day + 1;
```

```
    END LOOP;
  END LOOP;
  COMMIT;
END;
/
```

For a tournament with four registered teams, the procedure creates six fixtures. It can be executed using:

```
EXEC Generate_Fixtures(1, DATE '2026-11-01', 1);
```

## 10.4 Triggers

A trigger is a PL/SQL program that is automatically executed when a specified database event occurs. The first trigger sets the match status to "Scheduled" when a new match is inserted without a status. The second validates every new score entry: the scoring player must belong to one of the two teams playing the match, otherwise the entry is rejected.

```
CREATE OR REPLACE TRIGGER Match_Status_Trg
BEFORE INSERT ON Match
FOR EACH ROW
BEGIN
  IF :NEW.Match_Status IS NULL THEN
    :NEW.Match_Status := 'Scheduled';
  END IF;
END;
/
```

```
CREATE OR REPLACE TRIGGER Score_Validate_Trg
BEFORE INSERT ON Score
FOR EACH ROW
DECLARE
  V_Count NUMBER;
BEGIN
  SELECT COUNT(*) INTO V_Count
  FROM Player P JOIN Match M ON M.Match_ID = :NEW.Match_ID
  WHERE P.Player_ID = :NEW.Player_ID
  AND P.Team_ID IN (M.Home_Team_ID, M.Away_Team_ID);
  IF V_Count = 0 THEN
    RAISE_APPLICATION_ERROR(-20003,
      'Player does not belong to a team playing this match');
  END IF;
END;
/
```

PL/SQL functions, procedures, cursors, and triggers improve automation, reduce repeated operations, and help maintain consistency in the database.

# 11 File Organization and Indexing

File organization defines how records are stored and arranged in a database. Indexing is used to improve the speed of data retrieval by providing an efficient way to locate records. In the Sports Arena Management System, suitable file organization and indexing techniques help retrieve team, player, match, and score information efficiently.

## 11.1 File Organization

File organization refers to the method of arranging records in a database file. Common methods include sequential, heap, and indexed file organization. The system can use indexed organization for frequently searched attributes such as Team_ID, Player_ID, Match_ID, and Match_Date. This reduces the time required to locate records.

## 11.2 ISAM

Indexed Sequential Access Method (ISAM) stores records sequentially and maintains an index to provide faster access to specific records. It supports both sequential and direct access. In the Sports Arena Management System, ISAM can be used to efficiently access match or player records using their respective identifiers.

## 11.3 B+ Tree Indexing

A B+ Tree is a balanced tree structure commonly used for database indexing. It contains index keys in internal nodes and actual record pointers in leaf nodes. The leaf nodes are linked, which makes sequential and range searches efficient, such as finding all matches within a date range. For example, an index can be created on the Match table to speed up the venue availability check:

```
CREATE INDEX Match_Venue_Index ON Match(Venue_ID, Match_Date);
```

An index can also be created on the status of matches and on the player in the Score table, which speeds up the standings and player statistics queries:

```
CREATE INDEX Match_Status_Index ON Match(Match_Status);
CREATE INDEX Score_Player_Index ON Score(Player_ID);
CREATE INDEX Score_Match_Index ON Score(Match_ID);
```

B+ Tree indexing improves search performance and reduces the number of database blocks that need to be accessed. It is especially useful when the database contains a large number of match, player, and score records. Primary key and UNIQUE constraints automatically create indexes as well.

# 12 Testing

Testing is performed to verify that the Sports Arena Management System works correctly and produces the expected results. Different test cases are used to check normal operations, boundary conditions, and invalid inputs.

## 12.1 Test Cases

The following test cases are used to verify the main functions of the system.

Table 15: Test Cases for Sports Arena Management System

| Test ID | Test Case | Expected Result | Status |
|---|---|---|---|
| TC01 | Search for venues free on a date | Free venues are displayed | Pass |
| TC02 | Schedule a match (Schedule_Match) | Fixture is created with status Scheduled | Pass |
| TC03 | Schedule a match at a venue already booked that day | Fixture is rejected with an error message | Pass |
| TC04 | Cancel a valid match | Match status changes to Cancelled | Pass |
| TC05 | Insert a duplicate jersey number in the same team | Insert is rejected (UNIQUE violation) | Pass |
| TC06 | Insert a match with the same home and away team | Insert is rejected (CHECK violation) | Pass |
| TC07 | Enter scores and query Team_Standings | Standings show correct played, goal difference, and points | Pass |
| TC08 | Register a player with an invalid team ID | Appropriate error is displayed (FK violation) | Pass |

## 12.2 Normal and Boundary Cases

Normal cases test the system using valid inputs, such as scheduling a match at a free venue and registering a team with valid details. Boundary cases test values at the limits, such as a match on the first or last day of the tournament, two matches at the same venue on consecutive days, a goalless draw (no score entries, so each team earns one point), a player with jersey number 0 or 99, a scoring event in minute 0 or minute 120, and a match whose tickets sold equal the venue capacity.

### 12.3 Invalid Input Testing

Invalid input testing verifies that the system handles incorrect or unacceptable data properly. Examples include entering a non-existing Team ID, scheduling a match at a venue that is already booked for the same date, using a duplicate jersey number or email, entering a negative ticket price, recording a score for a player who is not in either team of the match, or attempting to use an invalid match status. The system should reject such inputs and maintain database consistency.

Testing ensures that the system performs its operations correctly and helps identify errors before the system is used in practice.

## 13 Reports

The Sports Arena Management System generates reports to provide useful information about fixtures, cancellations, revenue, and pending results. These reports help the organizers and administrator monitor the activities of the system and make better decisions.

### 13.1 Booking Report

The booking report provides details of the venue bookings made for matches. It includes the match ID, the home and away teams, the venue, the match date, and the match status.

```
SELECT M.Match_ID, H.Team_Name AS Home_Team, A.Team_Name AS Away_Team,
       V.Venue_Name, M.Match_Date, M.Match_Status
FROM Match M
JOIN Team H ON M.Home_Team_ID = H.Team_ID
JOIN Team A ON M.Away_Team_ID = A.Team_ID
JOIN Venue V ON M.Venue_ID = V.Venue_ID
ORDER BY M.Match_ID;
```

Table 16: Sample Booking Report

| Match ID | Home Team | Away Team | Venue | Date | Status |
|---|---|---|---|---|---|
| 401 | Falcons FC | Titans FC | Arena Stadium A | 2026-10-03 | Completed |
| 402 | Warriors United | Royals SC | Lakeside Sports Complex | 2026-10-04 | Completed |
| 403 | Falcons FC | Warriors United | Arena Stadium A | 2026-10-10 | Completed |
| 404 | Titans FC | Royals SC | Lakeside Sports Complex | 2026-10-09 | Scheduled |
| 405 | Falcons FC | Royals SC | Arena Stadium A | 2026-10-18 | Postponed |
| 406 | Titans FC | Warriors United | City Indoor Arena | 2026-10-20 | Cancelled |

## 13.2 Cancellation Report

The cancellation report displays the matches that have been cancelled or postponed. It helps track disrupted fixtures and keep accurate schedule records.

```
SELECT M.Match_ID, H.Team_Name AS Home_Team, A.Team_Name AS Away_Team,
       M.Match_Date, M.Match_Status
FROM Match M
JOIN Team H ON M.Home_Team_ID = H.Team_ID
JOIN Team A ON M.Away_Team_ID = A.Team_ID
WHERE M.Match_Status IN ('Cancelled','Postponed');
```

Table 17: Sample Cancellation Report

| Match ID | Home Team | Away Team | Match Date | Status |
|---|---|---|---|---|
| 405 | Falcons FC | Royals SC | 2026-10-18 | Postponed |
| 406 | Titans FC | Warriors United | 2026-10-20 | Cancelled |

## 13.3 Revenue Report

The revenue report provides information about the ticket income generated from completed matches at each venue. It can be used by the organizers to analyze the financial performance of the arena.

```
SELECT V.Venue_Name,
       COUNT(M.Match_ID) AS Matches_Held,
       SUM(M.Tickets_Sold) AS Tickets_Sold,
       SUM(M.Ticket_Price * M.Tickets_Sold) AS Revenue
FROM Match M
JOIN Venue V ON M.Venue_ID = V.Venue_ID
WHERE M.Match_Status = 'Completed'
GROUP BY V.Venue_Name;
```

Table 18: Sample Revenue Report

| Venue | Matches | Tickets Sold | Revenue |
|---|---|---|---|
| Arena Stadium A | 2 | 8,850 | Rs. 22,12,500 |
| Lakeside Sports Complex | 1 | 2,400 | Rs. 3,60,000 |

## 13.4 Overdue Rentals Report

The overdue report lists matches whose scheduled date has passed but which are still marked as Scheduled, that is, matches whose results have not yet been entered, so that the officials can be contacted.

```
SELECT M.Match_ID, H.Team_Name AS Home_Team, A.Team_Name AS Away_Team,
       V.Venue_Name, M.Match_Date
```

```
FROM Match M
JOIN Team H ON M.Home_Team_ID = H.Team_ID
JOIN Team A ON M.Away_Team_ID = A.Team_ID
JOIN Venue V ON M.Venue_ID = V.Venue_ID
WHERE M.Match_Status = 'Scheduled' AND M.Match_Date < TRUNC(SYSDATE);
```

For the sample data, this report lists match 404 (Titans FC vs Royals SC at Lakeside Sports Complex on 2026-10-09), whose score has not been entered. These reports provide a clear summary of fixtures, cancellations, revenue, and pending results, helping the organizers run the tournament and the arena efficiently.

# 14 Results and Discussion

The Sports Arena Management System successfully provides a structured database for managing tournaments, venues, teams, players, officials, matches, and scores. The implemented SQL commands allow records to be created, updated, deleted, and retrieved efficiently.

The database design reduces data redundancy through normalization up to 3NF (and BCNF) and maintains data integrity using primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints. Transactions and concurrency control help maintain consistency during fixture scheduling, score entry, and cancellation operations, and the availability check prevents clashing fixtures at the same venue or for the same team.

The implemented PL/SQL functions, procedures, cursors, and triggers provide additional automation, such as automatic fixture generation, score validation, and points calculation. Because the standings and player statistics are derived from the stored scores through views, the points table is always consistent with the recorded results. The testing results show that the system can handle normal operations, invalid inputs, clashing fixtures, and cancellations appropriately. The generated fixture, cancellation, revenue, and pending-result reports also provide useful information to the organizers.

Overall, the system meets the required database functionalities and provides a reliable and efficient approach for managing sports tournaments and arenas.

# 15 Conclusion

The Sports Arena Management System provides an efficient database-based solution for managing sports tournaments. The system organizes information related to tournaments, venues, teams, players, officials, matches, and scores in a structured manner, replacing manual score books and spreadsheets with a centralized relational database.

The project demonstrates important DBMS concepts including ER modeling, relational schema design, normalization, SQL DDL and DML commands, queries, views, transactions, concurrency control, PL/SQL functions, procedures, cursors, triggers, and indexing. These concepts help maintain data integrity, reduce redundancy, and improve the efficiency of database operations.

The testing and reports demonstrate that the system can handle team registration, fixture generation, score entry, standings, player statistics, and cancellations effectively. Overall, the proposed system provides a reliable and organized approach to sports tournament management and can be further enhanced with features such as online ticketing, live score updates, notifications, and user authentication.

# 16 SDG Mapping

The Sports Arena Management System contributes to the Sustainable Development Goals (SDGs) by promoting digitalization, efficient information management, and paperless services in sports administration.

Table 19: SDG Mapping of the Project

| **SDG Goal** | **Contribution of the Project** |
|---|---|
| SDG 3: Good Health and Well-being | The system makes it easier to organize leagues and tournaments, which encourages regular participation in sports and physical activity. |
| SDG 8: Decent Work and Economic Growth | The system helps arenas and sports organizers operate efficiently, tracks ticket revenue, and supports small sports enterprises in growing their services. |
| SDG 9: Industry, Innovation and Infrastructure | The system uses database technology to provide an efficient and reliable digital platform for managing sports facilities and competitions. |
| SDG 11: Sustainable Cities and Communities | Better scheduling improves the utilization of shared sports venues, giving communities fair access to public sports infrastructure, while digital records reduce paper usage. |

The project demonstrates how database technology can support digital and efficient services while reducing manual record maintenance and paper-based processes.

# 17 Future Scope

The Sports Arena Management System can be further enhanced with additional features to improve its functionality and user experience. Online ticketing and payment can be integrated to allow spectators to book seats digitally, and user authentication can be added to provide secure access for team managers, officials, and administrators.

The system can also be extended with live score updates, real-time standings on a mobile application, and email or SMS notifications for fixtures, postponements, and results. Support for knockout and group-stage formats, tie-break rules based on head-to-head results, player line-ups and injury records, referee performance tracking, sponsorship management, and automatic generation of match reports and certificates can also be introduced. These enhancements can make the system more secure, convenient, and suitable for real-world deployment.

# 18 Bibliography


1. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, Database System Concepts, McGraw-Hill Education.
2. Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, Pearson.
3. Oracle Corporation, Oracle Database Documentation.
4. MySQL Documentation, MySQL Reference Manual.
5. GeeksforGeeks, “DBMS and SQL Tutorials,” Online Reference.